Name of Student: Vyankatesh Rohokale	**PRN: 26070122518**
Semester: III	**Year : AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No : 24	
TITLE : JDBC with GUI	

Problem Statement

Develop a Java GUI application integrated with JDBC to perform database operations."

Learning Objectives

To integrate Java GUI applications with databases using JDBC.

Programme

1: Create a Library Management System GUI connected with database

```
import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class LibraryManagementSystem extends JFrame {

    JTextField idField;
    JTextField titleField;
    JTextField authorField;
    JTextField priceField;
    JTable table;
    DefaultTableModel model;

    Connection con;

    public LibraryManagementSystem() {

        setTitle("Library Management System");
        setSize(800, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
```

```
        connectDatabase();

        JPanel formPanel = new JPanel(new GridLayout(4, 2, 10,
10));

        formPanel.add(new JLabel("Book ID:"));
        idField = new JTextField();
        formPanel.add(idField);

        formPanel.add(new JLabel("Title:"));
        titleField = new JTextField();
        formPanel.add(titleField);

        formPanel.add(new JLabel("Author:"));
        authorField = new JTextField();
        formPanel.add(authorField);

        formPanel.add(new JLabel("Price:"));
        priceField = new JTextField();
        formPanel.add(priceField);

        JButton addButton = new JButton("Add Book");
        JButton updateButton = new JButton("Update Book");
        JButton deleteButton = new JButton("Delete Book");
        JButton clearButton = new JButton("Clear");

        JPanel buttonPanel = new JPanel();

        buttonPanel.add(addButton);
        buttonPanel.add(updateButton);
        buttonPanel.add(deleteButton);
        buttonPanel.add(clearButton);

        model = new DefaultTableModel(
                new String[]{"Book ID", "Title", "Author",
"Price"}, 0
        );

        table = new JTable(model);

        JScrollPane scrollPane = new JScrollPane(table);

        addButton.addActionListener(e -> addBook());
        updateButton.addActionListener(e -> updateBook());
        deleteButton.addActionListener(e -> deleteBook());
        clearButton.addActionListener(e -> clearFields());
```

```
        table.getSelectionModel().addListSelectionListener(e
-> {
            int row = table.getSelectedRow();

            if (row >= 0) {
                idField.setText(model.getValueAt(row,
0).toString());
                titleField.setText(model.getValueAt(row,
1).toString());
                authorField.setText(model.getValueAt(row,
2).toString());
                priceField.setText(model.getValueAt(row,
3).toString());
            }
        });

        JPanel topPanel = new JPanel(new BorderLayout(10,
10));
        topPanel.setBorder(BorderFactory.createEmptyBorder(10,
10, 10, 10));
        topPanel.add(formPanel, BorderLayout.CENTER);
        topPanel.add(buttonPanel, BorderLayout.SOUTH);

        add(topPanel, BorderLayout.NORTH);
        add(scrollPane, BorderLayout.CENTER);

        loadBooks();

        setVisible(true);
    }

    void connectDatabase() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/employee",
                    "root",
                    "admin@123"
            );

        } catch (Exception e) {
            JOptionPane.showMessageDialog(
                    this,
                    "Database connection failed: " +
e.getMessage()
            );
        }
    }
```

```
    void addBook() {

        try {
            String query = "INSERT INTO books VALUES (?, ?, ?,
?)";

            PreparedStatement ps =
con.prepareStatement(query);

            ps.setInt(1, Integer.parseInt(idField.getText()));
            ps.setString(2, titleField.getText());
            ps.setString(3, authorField.getText());
            ps.setDouble(4,
Double.parseDouble(priceField.getText()));

            ps.executeUpdate();

            JOptionPane.showMessageDialog(this, "Book added
successfully");

            ps.close();

            clearFields();
            loadBooks();

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error: " +
e.getMessage());
        }
    }

    void updateBook() {

        try {
            String query = "UPDATE books SET title = ?, author
= ?, price = ? WHERE book_id = ?";

            PreparedStatement ps =
con.prepareStatement(query);

            ps.setString(1, titleField.getText());
            ps.setString(2, authorField.getText());
            ps.setDouble(3,
Double.parseDouble(priceField.getText()));
            ps.setInt(4, Integer.parseInt(idField.getText()));

            int rows = ps.executeUpdate();
```

```
            if (rows > 0) {
                JOptionPane.showMessageDialog(this, "Book
updated successfully");
            } else {
                JOptionPane.showMessageDialog(this, "Book not
found");
            }

            ps.close();

            clearFields();
            loadBooks();

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error: " +
e.getMessage());
        }
    }

    void deleteBook() {

        try {
            String query = "DELETE FROM books WHERE book_id
= ?";

            PreparedStatement ps =
con.prepareStatement(query);

            ps.setInt(1, Integer.parseInt(idField.getText()));

            int rows = ps.executeUpdate();

            if (rows > 0) {
                JOptionPane.showMessageDialog(this, "Book
deleted successfully");
            } else {
                JOptionPane.showMessageDialog(this, "Book not
found");
            }

            ps.close();

            clearFields();
            loadBooks();

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error: " +
e.getMessage());
        }
```

```
    }

    void loadBooks() {

        try {
            model.setRowCount(0);

            String query = "SELECT * FROM books";

            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(query);

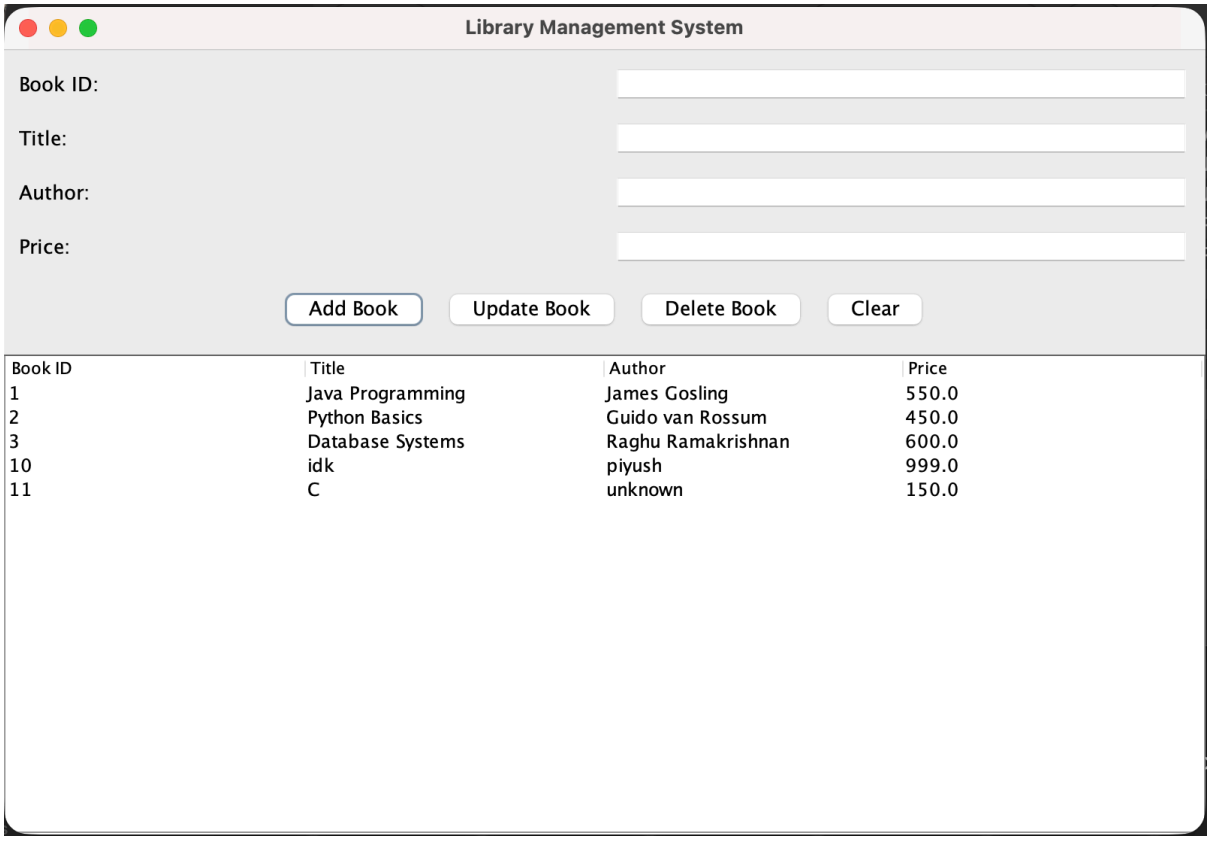
            while (rs.next()) {
                model.addRow(new Object[]{
                        rs.getInt("book_id"),
                        rs.getString("title"),
                        rs.getString("author"),
                        rs.getDouble("price")
                });
            }

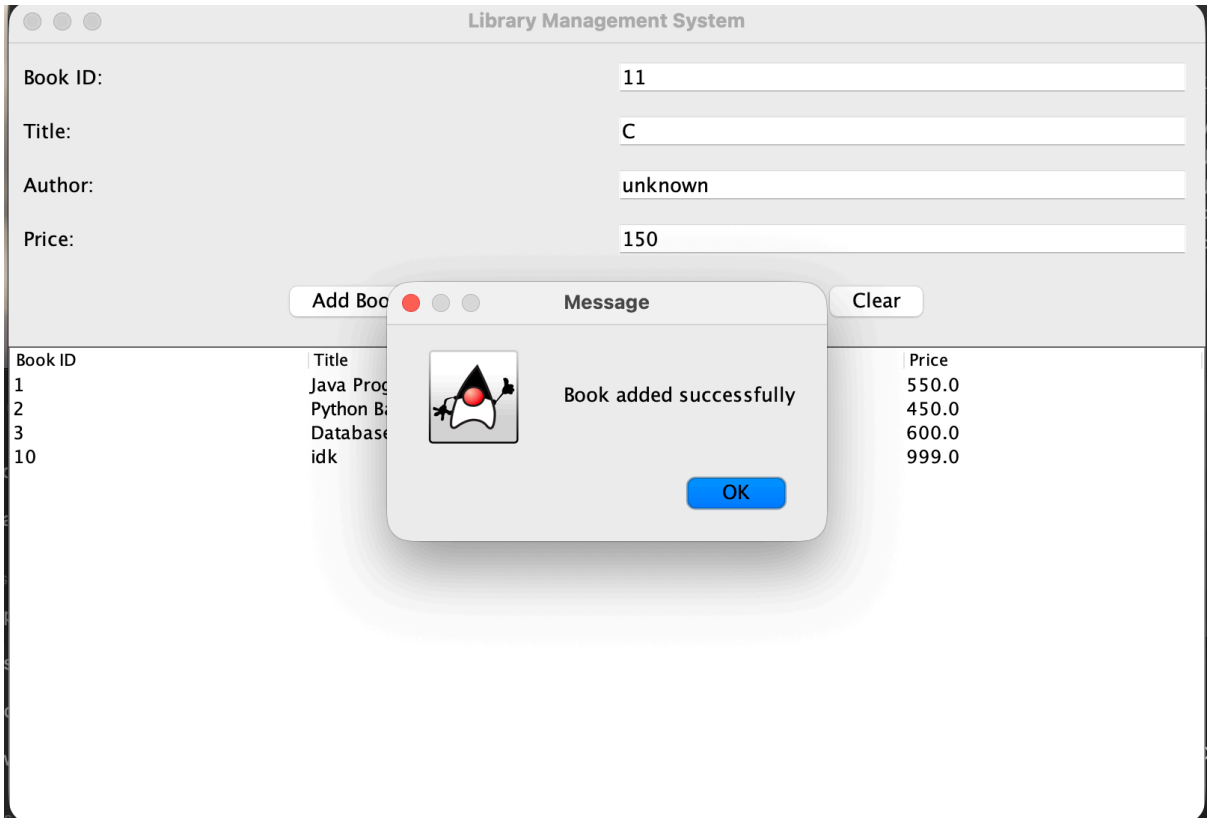
            rs.close();
            stmt.close();

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error: " +
e.getMessage());
        }
    }

    void clearFields() {
        idField.setText("");
        titleField.setText("");
        authorField.setText("");
        priceField.setText("");
    }

    public static void main(String[] args) {
        new LibraryManagementSystem();
    }
}
```


Library Management System
Book ID:
Title:
Author:
Price:
Add Book
Update Book
Delete Book
Clear
Book ID Title Author Price
1 Java Programming James Gosling 550.0
2 Python Basics Guido van Rossum 450.0
3 Database Systems Raghu Ramakrishnan 600.0
10 idk piyush 999.0
11 C unknown 150.0


Library Management System
Book ID: 11
Title: C
Author: unknown
Price: 150
Add Boo
Clear
Message
Book added successfully
OK
Book ID Title Price
1 Java Prog 550.0
2 Python Ba 450.0
3 Database 600.0
10 idk 999.0

2: Create a Book Issue Tracking System GUI application using Java Swing connected to a database. The system should store book ID, student name, issue date, and return date, and allow record management.

```
import java.awt.*;
import java.sql.*;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;

public class BookIssueTracking extends JFrame {

    JTextField bookIdField;
    JTextField studentNameField;
    JTextField issueDateField;
    JTextField returnDateField;

    JTable table;
    DefaultTableModel model;

    Connection con;

    public BookIssueTracking() {

        setTitle("Book Issue Tracking System");
        setSize(850, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        connectDatabase();

        JPanel formPanel = new JPanel(new GridLayout(4, 2, 10,
10));

        formPanel.add(new JLabel("Book ID:"));
        bookIdField = new JTextField();
        formPanel.add(bookIdField);

        formPanel.add(new JLabel("Student Name:"));
        studentNameField = new JTextField();
        formPanel.add(studentNameField);

        formPanel.add(new JLabel("Issue Date (YYYY-MM-DD):"));
        issueDateField = new JTextField();
        formPanel.add(issueDateField);
```

```
        formPanel.add(new JLabel("Return Date (YYYY-MM-
DD):"));
        returnDateField = new JTextField();
        formPanel.add(returnDateField);

        JButton addButton = new JButton("Add Record");
        JButton updateButton = new JButton("Update Record");
        JButton deleteButton = new JButton("Delete Record");
        JButton clearButton = new JButton("Clear");

        JPanel buttonPanel = new JPanel();

        buttonPanel.add(addButton);
        buttonPanel.add(updateButton);
        buttonPanel.add(deleteButton);
        buttonPanel.add(clearButton);

        model = new DefaultTableModel(
                new String[]{"Book ID", "Student Name", "Issue
Date", "Return Date"}, 0
        );

        table = new JTable(model);

        JScrollPane scrollPane = new JScrollPane(table);

        addButton.addActionListener(e -> addRecord());
        updateButton.addActionListener(e -> updateRecord());
        deleteButton.addActionListener(e -> deleteRecord());
        clearButton.addActionListener(e -> clearFields());

        table.getSelectionModel().addListSelectionListener(e
-> {

            int row = table.getSelectedRow();

            if (row >= 0) {
                bookIdField.setText(model.getValueAt(row,
0).toString());
                studentNameField.setText(model.getValueAt(row,
1).toString());
                issueDateField.setText(model.getValueAt(row,
2).toString());
                returnDateField.setText(model.getValueAt(row,
3).toString());
            }
        });
```

```
        JPanel topPanel = new JPanel(new BorderLayout(10,
10));

        topPanel.setBorder(
                BorderFactory.createEmptyBorder(10, 10, 10,
10)
        );

        topPanel.add(formPanel, BorderLayout.CENTER);
        topPanel.add(buttonPanel, BorderLayout.SOUTH);

        add(topPanel, BorderLayout.NORTH);
        add(scrollPane, BorderLayout.CENTER);

        loadRecords();

        setVisible(true);
    }

    void connectDatabase() {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            con = DriverManager.getConnection(
                    "jdbc:mysql://localhost:3306/employee",
                    "root",
                    "admin@123"
            );

        } catch (Exception e) {

            JOptionPane.showMessageDialog(
                    this,
                    "Database connection failed: " +
e.getMessage()
            );
        }
    }

    void addRecord() {

        try {

            String query = "INSERT INTO book_issue VALUES
(?, ?, ?, ?)";
```

```
            PreparedStatement ps =
con.prepareStatement(query);

            ps.setInt(1,
Integer.parseInt(bookIdField.getText()));
            ps.setString(2, studentNameField.getText());
            ps.setDate(3,
Date.valueOf(issueDateField.getText()));
            ps.setDate(4,
Date.valueOf(returnDateField.getText()));

            ps.executeUpdate();

            JOptionPane.showMessageDialog(
                    this,
                    "Book issue record added successfully"
            );

            ps.close();

            clearFields();
            loadRecords();

        } catch (Exception e) {

            JOptionPane.showMessageDialog(
                    this,
                    "Error: " + e.getMessage()
            );
        }
    }

    void updateRecord() {

        try {

            String query =
                    "UPDATE book_issue SET student_name = ?,
issue_date = ?, return_date = ? WHERE book_id = ?";

            PreparedStatement ps =
con.prepareStatement(query);

            ps.setString(1, studentNameField.getText());
            ps.setDate(2,
Date.valueOf(issueDateField.getText()));
            ps.setDate(3,
Date.valueOf(returnDateField.getText()));
```

```
            ps.setInt(4,
Integer.parseInt(bookIdField.getText()));

            int rows = ps.executeUpdate();

            if (rows > 0) {

                JOptionPane.showMessageDialog(
                        this,
                        "Book issue record updated
successfully"
                );

            } else {

                JOptionPane.showMessageDialog(
                        this,
                        "Record not found"
                );
            }

            ps.close();

            clearFields();
            loadRecords();

        } catch (Exception e) {

            JOptionPane.showMessageDialog(
                    this,
                    "Error: " + e.getMessage()
            );
        }
    }

    void deleteRecord() {

        try {

            String query =
                    "DELETE FROM book_issue WHERE book_id
= ?";

            PreparedStatement ps =
con.prepareStatement(query);

            ps.setInt(
                    1,
                    Integer.parseInt(bookIdField.getText())
```

```
            );

            int rows = ps.executeUpdate();

            if (rows > 0) {

                JOptionPane.showMessageDialog(
                        this,
                        "Book issue record deleted
successfully"
                );

            } else {

                JOptionPane.showMessageDialog(
                        this,
                        "Record not found"
                );
            }

            ps.close();

            clearFields();
            loadRecords();

        } catch (Exception e) {

            JOptionPane.showMessageDialog(
                    this,
                    "Error: " + e.getMessage()
            );
        }
    }

    void loadRecords() {

        try {

            model.setRowCount(0);

            String query = "SELECT * FROM book_issue";

            Statement stmt = con.createStatement();

            ResultSet rs = stmt.executeQuery(query);

            while (rs.next()) {

                model.addRow(
```

```
                        new Object[]{
                                rs.getInt("book_id"),
                                rs.getString("student_name"),
                                rs.getDate("issue_date"),
                                rs.getDate("return_date")
                        }
                );
            }

            rs.close();
            stmt.close();

        } catch (Exception e) {

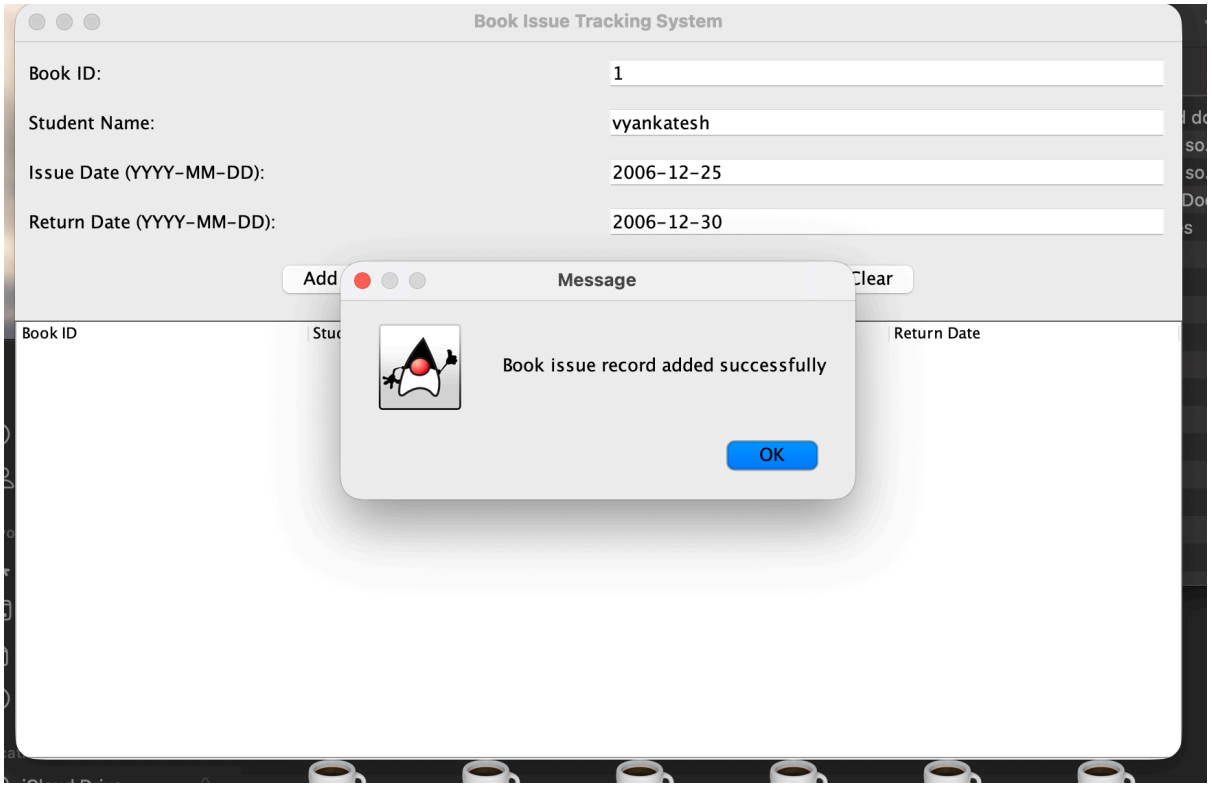
            JOptionPane.showMessageDialog(
                    this,
                    "Error: " + e.getMessage()
            );
        }
    }

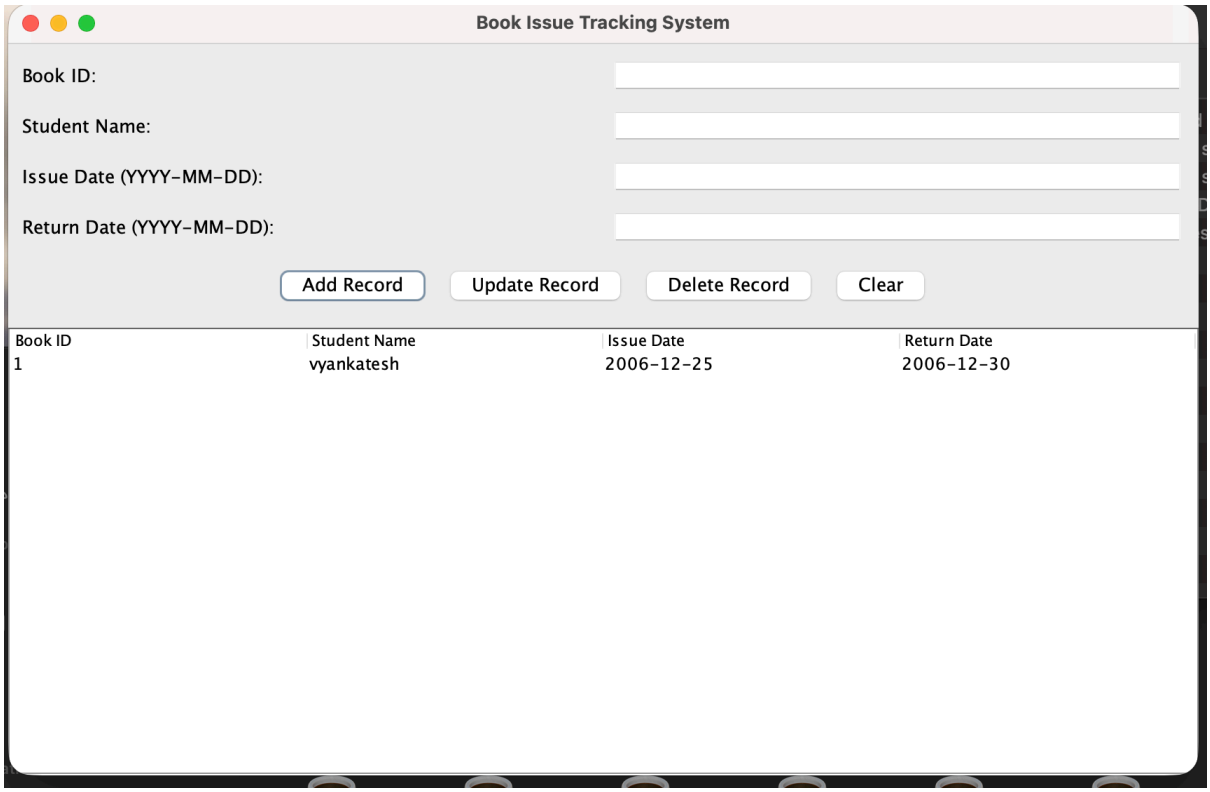
    void clearFields() {

        bookIdField.setText("");
        studentNameField.setText("");
        issueDateField.setText("");
        returnDateField.setText("");
    }

    public static void main(String[] args) {

        new BookIssueTracking();
    }
}
```





Conclusion :

The Book Issue Tracking System successfully demonstrates a Java Swing GUI connected to a MySQL database using JDBC. The application allows users to add, view, update, and delete book issue records containing book ID, student name, issue date, and return date.

Github :


Java_assignments / assignment24 /
Add file
VyankateshRohokale assignment 24
185359c · now
History
Name
Last commit message
Last commit date
..
BookIssueTracking.java
assignment 24
now
LibraryManagementSystem.java
assignment 24
now